

Don't Let the Model Write the Query

Software architecture for repeatable chat responses over SQL data

John Ulett · SageTechTN Vibe Coder

`the model chooses` · `the code composes`

PeopleIQ - The domain, once

- **A recruiting firm**

Two source systems. A decade of history.
CRM with 17k companies + 23k people
ATS with 23k people

- **Non-technical users**

Questions typed in plain English.

- **Real decisions**

Hiring and business development ride on the answers.

- **~25 tables in scope**

Small enough to reason about. Big enough to get wrong.

Repeatability is not accuracy

WHAT BENCHMARKS MEASURE

Is the generated SQL correct?

WHAT PRODUCTION NEEDS

Is it the same SQL every time?

Lose the second one and you lose all four:

can't cache it

can't test it

can't audit it

can't explain it

"I'd rather be 85% right the same way every time than 95% right in a way I can't reproduce."

Three ways to build this

Text-to-SQL

THE MODEL RETURNS

Any string

FAILURE MODE

Confidently wrong SQL, differently each time

Semantic match over a query library

THE MODEL RETURNS

A retrieved snippet

FAILURE MODE

Nearest neighbour $\neq$ right answer shape

Classify the row unit, then compose

THE MODEL RETURNS

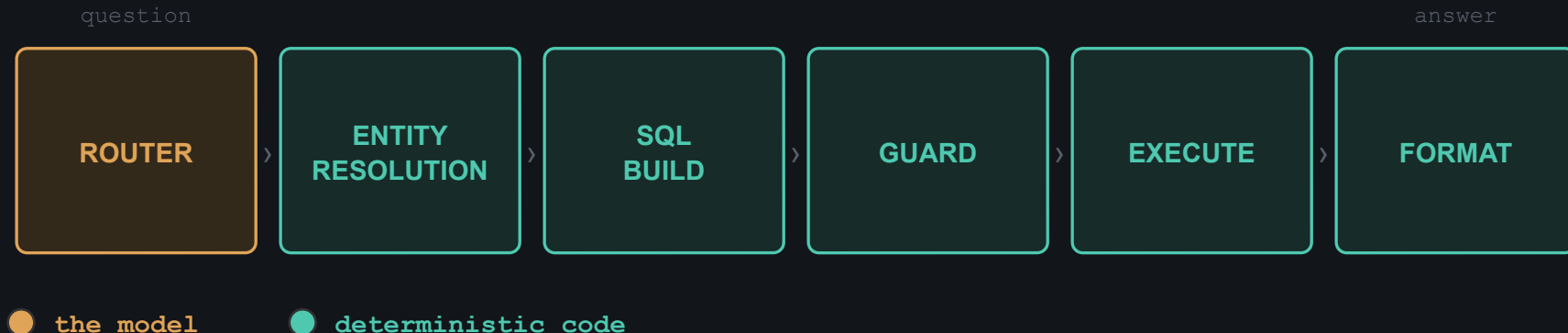
A typed struct

FAILURE MODE

Confident mis-routing — see slide 15

The middle one matches on phrasing. The third asks a different question entirely.

One model call, then deterministic code



The router is not choosing a query.

It answers one question first: **what is one row of the answer?**

A search? A person? A candidate in a pipeline? An aggregate bucket?

The blast radius, written down

```
class RouterDecision(BaseModel):  
    intent: Literal["query", "expansion", "clarify_response", "chitchat"]  
    template_id: Optional[str] = None  
    question_type: Optional[Literal["count", "list",  
                                    "table", "summary", "scalar"]] = None  
    parameters: list[RouterParam] = Field(...)  
    expansions: list[str] = Field(...)  
    confidence: float = 0.0  
    clarifying_question: Optional[str] = None  
    limit: Optional[int] = None  
    sort: Optional[str] = None
```

9

fields, total

4

closed enumerations

0

ways to name a table

Templates are data, not code

```
base_sql: |
  SELECT {columns} FROM thrive_searches s
  WHERE 1=1 {filters}
  ORDER BY {order_by} LIMIT {limit}

filters:
  client: {sql: "AND s.company_name = %(client)s"}
  status: {sql: "AND s.status = %(status)s"}
  title: {sql: "AND s.job_title LIKE %(title)s"}
```

A config row in the database — not a file in the codebase.
New capability = an INSERT, not a deploy.

User values are never interpolated — only bound.

That %(client)s stays a placeholder all the way into the driver.

Structural SQL is never model-authored.

The model picks a sort KEY. A human wrote the ORDER BY behind it.

“But templates don’t scale”

16,384

filter combinations from
14 freely-combining filters

× 5

result shapes — count, list, table, summary, scalar

× 2^{13}

subsets of 13 optional columns

×

sort key, row limit, and follow-up state

One hand-written SQL frame.

The question space is infinite. The **shape** space is small — and it stops growing.

Where domain knowledge goes to live

The user asked for "VP of Operations". The query became **%VP of Operations%**.
It matched **zero** live searches.

The real titles: Market VP Operations · SVP Operations (IDD) · COO/VP Operations

Everything worked.

Question clear. Routing correct. SQL valid. Answer empty.

The fix was ten lines.

Drop stopwords, match significant words in order.

It went in once.

Every job-title query in the system got permanently better.

If a model writing SQL fresh every call has nowhere to put that fix — except an ever-growing prompt.

Solved with PeopleIQ Framework.

The question in, the struct out

“open searches for Northwind with recruiter and candidate counts”

▼ everything the model said

```
{ "intent": "query", "template_id": "template_search_01",  
  "question_type": "table", "confidence": 0.94,  
  "parameters": [{ "name": "client", "value": "Northwind" },  
                  { "name": "status", "value": "Open" } ],  
  "expansions": [ "recruiter", "candidate_count" ] }
```

It translated “open” through a business glossary, and turned “with recruiter and candidate counts” into two column keys — not into SQL. Into two strings that must match keys the template declares.

The SQL the code built

```
SELECT s.name AS `Search`, s.company_name AS `Client`,  
       s.assigned_to_name AS `Recruiter`,  
       s.candidacies_count AS `Candidates`  
FROM thrive_searches s  
WHERE 1=1 AND s.company_name = %(client)s  
       AND s.status = %(status)s  
ORDER BY s.created_at DESC LIMIT 100
```

The model said “Northwind”. Code resolved that to “**Northwind Health**” by fuzzy match.

The user typed a company name. That text is still not in the SQL string.

There is no injection surface here — not because we sanitized well, but because **user text never becomes SQL text.**

The guard

- 1 Exactly one statement, and it must be a SELECT
- 2 Every table on the allow-list
- 3 No restricted column in the projection
- 4 LIMIT injected when missing, clamped when too high

THE SUBTLETY

The restriction is on what reaches the **user** — not what the query **touches**.

A foreign key in a JOIN is fine. The same column in the SELECT list is rejected.

The same guard runs on template SQL and on model-written SQL.

The trust boundary is in code, after the model — never in the prompt.

Ask it again tomorrow

TODAY

```
SELECT s.name, s.company_name,  
       s.assigned_to_name, ...  
FROM thrive_searches s  
WHERE 1=1 AND ... = %(client)s  
ORDER BY s.created_at DESC
```

NEXT TUESDAY

```
SELECT s.name, s.company_name,  
       s.assigned_to_name, ...  
FROM thrive_searches s  
WHERE 1=1 AND ... = %(client)s  
ORDER BY s.created_at DESC
```

byte for byte

```
key = (template_id, parameters, expansions, variant)
```

Cacheable on that key — and the key only works because the query is a pure function of it. Caching isn't a bonus feature. It's **evidence** the architecture is what I claim.

The 5% you can't template

1

Confidence below threshold

SQL written from a schema description with forbidden tables and columns removed — absent, not merely prohibited.

2

Guarded identically

Same AST parse, same allow-list, same projection rules as everything else.

3

Logged and flagged

Every fallback lands in the query log with its confidence and its reason.

Every flagged fallback is a template you haven't written yet.

The fallback rate is a KPI, and it should trend toward zero. Your escape hatch should also be your backlog.

What this costs you

Coverage grows linearly with human effort

A new result shape means someone writes SQL. That is a real ceiling.

The template library is a second schema

Kept in sync with the real one by hand.

Confident mis-routing is the scary failure

Bad generated SQL errors loudly. A wrong template returns a beautiful, correctly formatted table answering a question nobody asked.

Config-as-data has no type checker

A malformed template row degrades quietly rather than failing the build.

Two mitigations: thresholds shouldn't be cliffs, and "ask a clarifying question" is a valid output.

Three things, none about SQL

1

Constrain the model's output surface, not its input.

A nine-field struct beats any amount of prompt engineering.

2

Put the trust boundary in code, after the model.

Write it as though the prompt already failed. Sometimes it has.

3

Make the escape hatch measurable.

The thing you fall back to should tell you what to build next.

The model chooses. The code composes.